PASETO
Platform-Agnostic Security Tokens

PASETO (Platform-Agnostic Security Tokens) es un estándar para la creación de **tokens de seguridad** utilizados para transportar información entre sistemas de forma protegida.

→ Fue diseñado con un enfoque más prescriptivo en criptografía, reduciendo decisiones de configuración que pueden provocar implementaciones inseguras.

PASETO significa **Platform-Agnostic Security Tokens**. Es un estándar para crear **tokens de seguridad** que permiten transportar información —claims— de forma protegida, especialmente en procesos de **autenticación y autorización**.

Un token PASETO puede contener información o **claims** como:

```
{
  "sub": "1",
  "email": "admin@hospital.com",
  "rol": "administrador"
}
```

El proyecto de referencia trabaja con:

```
v4.public
 |    |
 |    └── Propósito: autenticidad e integridad
 |
 └── Versión 4 del protocolo
```

```
Usuario
   ↓
POST /login
   ↓
email + password
   ↓
¿Credenciales correctas?
   ↓
Generamos PASETO
   ↓
v4.public.xxxxxxxxx...
   ↓
Cliente guarda el token
   ↓
Authorization: Bearer <token>
   ↓
GET /perfil
   ↓
Verificamos PASETO
   ↓
Acceso permitido
```

En nuestro caso estamos trabajando con **PASETO v4.public**. Aquí se utiliza criptografía asimétrica:

CLAVE PRIVADA
↓
FIRMA
↓
TOKEN PASETO
↓
v4.public...
↓
VERIFICACIÓN
↓
CLAVE PÚBLICA

PASETO = Platform-Agnostic Security Tokens
v4 = versión del protocolo
public = token firmado con criptografía asimétrica
clave privada = firma
clave pública = verifica.

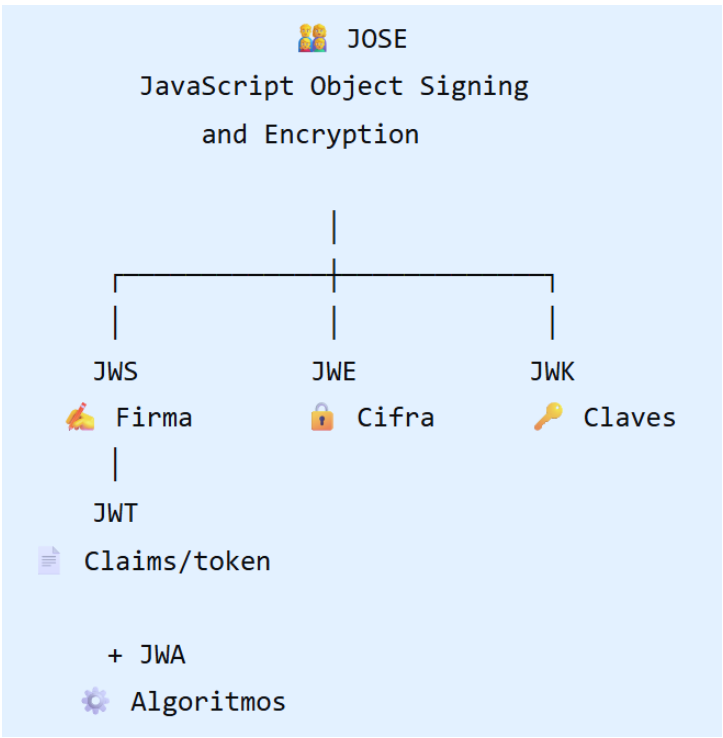
PASETO es un estándar de tokens de seguridad, agnóstico a la plataforma, diseñado para autenticación y autorización mediante opciones criptográficas definidas por versión y propósito.

JavaScript Object Signing and Encryption - JOSE

Firma y cifrado de objetos JavaScript

Sigla	Significado	Función
JWT	JSON Web Token	Representar claims en un token
JWS	JSON Web Signature	Firmar datos
JWE	JSON Web Encryption	Cifrar datos
JWK	JSON Web Key	Representar claves criptográficas
JWA	JSON Web Algorithms	Definir algoritmos criptográficos

JOSE no es una única tecnología, sino un conjunto de estándares relacionados que trabajan juntos. Por eso se habla de la "familia JOSE".



Cada miembro de esta familia tiene una responsabilidad diferente, pero están diseñados para relacionarse.

Por ejemplo, JWT responde principalmente a:

¿Cómo representamos un conjunto de claims en un token?

```
{
 "sub": "123",
 "rol": "administrador"
}
```

JWS responde:

¿Cómo protegemos su integridad mediante una firma?

JWE:

¿Cómo protegemos su confidencialidad mediante cifrado?

JWK:

¿Cómo representamos las claves criptográficas en JSON?

JWA:

¿Qué algoritmos criptográficos están definidos para utilizarse en estos mecanismos?

Por eso no sería del todo preciso decir:

JOSE = JWT

Es mejor pensar de esta manera:

```
JOSE
 │
 ├── JWT → tokens/claims
 ├── JWS → firma
 ├── JWE → cifrado
 ├── JWK → claves
 └── JWA → algoritmos
```

JWT y PASETO son alternativas para construir sistemas de tokens.

JWT tiene una adopción y un ecosistema enormes; PASETO busca proporcionar un diseño más prescriptivo que reduzca ciertas decisiones criptográficas propensas a errores.

La elección entre JWT y PASETO depende del ecosistema, interoperabilidad, requisitos criptográficos, bibliotecas disponibles y arquitectura del sistema; no de que uno sea universalmente "bueno" y el otro "malo".

JWT	PASETO
JSON Web Token	Platform-Agnostic Security Tokens
Muy extendido	Alternativa moderna
Familia JOSE	Diseño propio de PASETO
Varias combinaciones criptográficas	Versiones y propósitos más definidos
Ej. JWT firmado	Ej. v4.public
Claims	Claims

Una metáfora que funciona muy bien para entender **JOSE, JWT, JWS, JWE y JWK** es pensar en el **servicio de correspondencia de una empresa**.

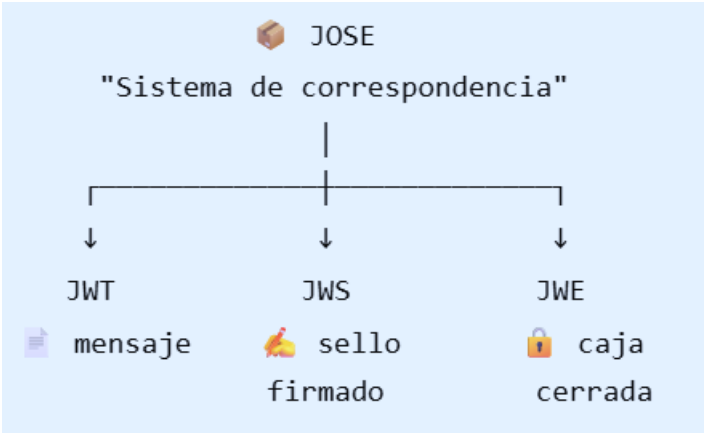
La metáfora del paquete

Imaginemos que queremos enviar este mensaje:

Usuario: juan
Rol: administrador
ID: 123

→ Ese mensaje son nuestros **claims**.

Ahora tenemos varias formas de enviarlo.



JWT sería el documento que queremos transportar. Contiene la información:

```
{
  "sub": "123",
  "nombre": "Juan",
  "rol": "administrador"
}
```

Pero necesitamos decidir cómo protegerlo.

JWS sería como poner el documento dentro de un sobre transparente y agregarle un sello oficial.

Usuario: Juan
Rol: administrador
ID: 123
✍ SELLO OFICIAL

La gente puede leer el contenido, pero si alguien cambia:

rol: administrador

por otra cosa, el sello deja de ser válido.

Por eso:

JWS protege integridad y autenticidad, no confidencialidad.

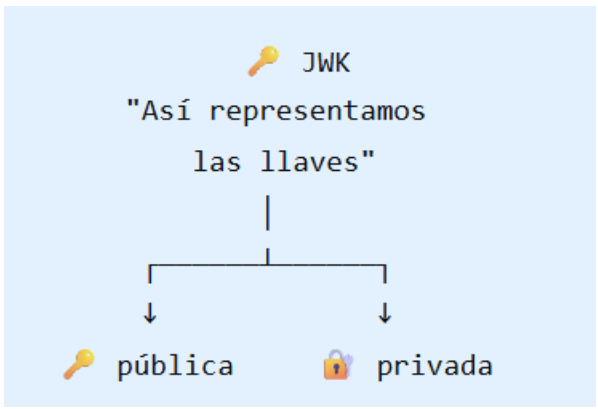
Ahora imaginemos que esa información es confidencial. Utilizamos **JWE**, que sería como colocar el documento dentro de una **caja fuerte cerrada**:



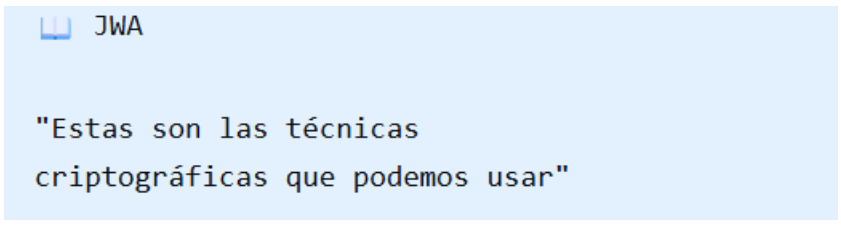
Quien intercepte el paquete no puede simplemente leer el contenido.

¿Y JWK?

Es como el **registro de las llaves** que utilizamos:



Y **JWA** sería el catálogo de mecanismos autorizados:



Podemos percibirlo e interpretar **JOSE** así:



¿Y dónde entra PASETO?

JOSE sería una empresa que te dice:

→ "Aquí tienes diferentes sobres, cajas, sellos, llaves y mecanismos. **Elige y configúralos correctamente**."

Mientras **PASETO** se parece más a un servicio que dice:

→ "No quiero que tengas tantas combinaciones criptográficas que elegir. Dime qué propósito necesitas y usa el protocolo definido para esa versión."



↗ **JOSE** es como un sistema de correspondencia con **muchas herramientas y opciones**
↗ **PASETO** intenta entregar un **procedimiento más definido** para evitar combinaciones criptográficas inadecuadas.